

Code-Layout, Readability and Reusability

Date	03 October 2026
Team ID	SWTID-2026-5999
Project Name	PocketSmart AI — AI-Assisted Goal-Based Budget Planner
Maximum Marks	5 Marks

Code Layout Checklist

S.No	Parameter	Description	Status	Evidence
1	File organization	Authentication, models, database, AI service and routes are separated	Yes	auth.py, models.py, database.py, gemini_utils.py, main.py
2	Naming	Functions and variables describe their purpose	Yes	get_home_recommendations, build_shopping_links, remaining_budget
3	Validation	Common input rules are defined in Pydantic models	Yes	models.py
4	Reusable AI logic	One Gemini service handles model calls and fallback	Yes	gemini_utils.py
5	Reusable budget logic	Common calculation and trimming helpers are shared	Yes	_finalize_breakdown, _trim_to_budget
6	Shared UI	Common page structure is inherited from a base template	Yes	base.html, planner_base.html
7	Error handling	Invalid requests and external AI failures are handled	Yes	FastAPI validation and fallback paths
8	Configuration	Secrets and environment-specific values are not hard-coded	Yes	.env.example, config.py

Reusable Components

Component	Used In	Reuse
build_shopping_links()	Home, Party and Jewelry	High
_finalize_breakdown()	All planners	High
_trim_to_budget()	All planners	High
Gemini model chain	All AI requests	High
base.html	Main application pages	High
Auth dependency	Protected routes	High
JavaScript result renderer	Planner results and history	High

Readability Approach

The code favours small functions with a single purpose. Comments explain decisions that are not obvious from the code, while the phase documents explain the system at a higher level. This makes it easier for a new team member to locate a feature without reading the whole application.